CRYPTO INSANE (500 PTS)

Eightfold Funeral: Epitaph

This was a heavy 500-point crypto problem split into two distinct parts: breaking an obfuscated affine stamp map, then solving an 8-sum modular collision problem over 2^{84}.

Phase 1: Reversing the Stamp Map

`model.py` generates stamps via an affine transformation followed by a bitwise circular rotation over 84 bits: `stamp = ROL84((a * x + b) % 2^84, r)` We're given 25 calibration input/stamp pairs. With only 84 possible rotation values ($r \in [0, 83]$), we don't need fancy math to find r: 1. For each candidate rotation $r \in [0, 83]$, rotate all calibration stamps back by r. 2. Pick any two points (x_1, y_1) and (x_2, y_2). If $\Delta x = x_1 - x_2$ is odd, it's coprime to 2^{84}, so we can compute its modular inverse and solve for a and b directly: `a = ((y1 - y2) * inverse(x1 - x2, 2^84)) % 2^84` `b = (y1 - a * x1) % 2^84` 3. Testing candidates against all calibration pairs revealed that 15 pairs agreed on the exact same parameters (the other 10 were decoys):

- $r = 35$
- $a = \text{0x750e59bef1f0e6b385693}$
- $b = \text{0x67cd6060c7616d28d7c34}$

Inverting this map on `lane_stamps` unlocked the 8 lane seeds, and applying it to `target_stamp` revealed the target sum: `0x1b59868b301e63f23871f`.

Phase 2: Wagner's Generalized Birthday Attack (The 8-Sum)

Each of the 8 lanes generates 2^{21} labels (derived from SHA-256). We need to pick one label from each lane such that their sum equals the target modulo 2^{84}: $\sum_{i=0}^{7} L_i[idx_i] \equiv \text{target} \pmod{2^{84}}$
A naive search over 8×2^{21} possibilities is $O(2^{168})$, which is impossible. But notice the numbers: $8 = 2^3$ lists, and $84 \text{ bits} / 4 = 21 \text{ bits}$. This is the ideal setup for **Wagner's k-tree generalized birthday attack**: 1. Subtract the target from lane 7 so we are hunting for an 8-sum that equals 0. 2. **Layer 1:** Pair adjacent lanes $(0,1), (2,3), (4,5), (6,7)$ and filter for partial sums where the lowest 21 bits are zero. Each list retains roughly 2^{21} surviving sums. 3. **Layer 2:** Pair those 4 lists into 2 lists, keeping sums where the next 21 bits (bits 21–41) are zero. Now we have two lists of 2^{21} values whose low 42 bits are strictly zero. 4. **Layer 3:** Sort both lists on their top 42 bits and look for a two-sum match that zeros out the remaining 42 bits.

Wrote the tree search in C++ with `unsigned __int128` and OpenSSL's SHA-256 for speed:

```
$ head -n 20 solve.cpp
```

```
❯ head -n 20 solve.cpp
#include <openssl/sha.h>
#include <algorithm>
#include <array>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <vector>

using U = unsigned __int128;
constexpr U MASK = ((U)1 << 84) - 1;
constexpr uint32_t N = 1 << 21;

struct E { U v; uint64_t t; };
uint64_t seeds[8][2] = {};
U target = 0;

int main() {
    printf("[+] Initializing Wagner generalized birthday attack (8-sum mod 2^84)...\n");
    printf("[+] Memory pool allocated for 8 lanes with 2^21 labels each.\n");
    printf("[+] Matching tree layer 1 -> layer 2 -> final 42-bit collision.\n");
```

C++ Wagner Attack Implementation

The solver hit the matching indices in about 40 seconds: `1924587 2031184 29597 785188 676753 560734 264902 1161509`

Phase 3: Unlocking the Capsule

`receipt_key()` hashes the indices and seeds into an AES-256 key. The ciphertext in `capsule` was AES-GCM, but since we didn't have the

original associated authenticated data (AAD) for the tag, we decrypted the ciphertext block directly via AES-CTR (counter block initialized to `nonce || 0x00000002`):

Solution Script: Eightfold_Funeral_Epitaph.py (included in attached solution archive)

Running the multi-phase solver:

```
$ python3 solve.py
```

```
❯ python3 solve.py
[Phase 1] Inverting 84-bit rotated affine stamp map...
  [+] Rotation r = 35
  [+] Multiplier a = 0x750e59bef1f0e6b385693
  [+] Addend b = 0x67cd6060c7616d28d7c34
  [+] Target 8-sum modulo 2^84: 0x1b59868b301e63f23871f

[Phase 2] Executing Wagners 3-layer generalized birthday attack...
  [+] Found collision indices across 8 lanes: [1924587, 2031184, 29597, 785188, 676753, 560734, 264902, 1161509]

[Phase 3] Decrypting AES capsule...
[+] Flag: SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}

~/SecLeaf writeups/repo_clone/SecLeafCTF/CRYPTO/Eightfold_Funeral_Epitaph main*
❯
```

Solver Execution Proof

Flag:

Solution Script: Eightfold_Funeral_Epitaph.py (included in attached solution archive)

RECOVERED FLAG `SecLeaf{m0dular_c4rr13s_d0nt_r3st_1n_p34c3}`